

Priority Engine Modernization Report

A rebuild of the complaint priority-scoring engine: every weight is now computed from a documented, citable methodology instead of an unexplained literal, the three previously conflicting scoring formulas were consolidated into one, and a confirmed architectural bug that discarded the engine's own weighted infrastructure score was fixed.

10 correctness & consistency fixes

4 AHP-derived weight sets

4 dead/duplicate files removed

8 files changed

Project: Urban Pulse — UrbanPulse_Backend

Scope: services/LocationPriorityService.js, services/EmotionAnalysisService.js, services/TextAuthenticityService.js, routes/complaints.js, routes/locationPriority.js, and two new files: services/ahp.js, services/priorityConfig.js

Date: September 3, 2026

1 What this report covers

A prior audit of Urban Pulse's complaint priority engine found two classes of problem: (a) a set of concrete correctness bugs where the code did not do what its own comments claimed, and (b) every numeric weight, threshold, and multiplier in the system was an unexplained literal with no documented source. This report covers the fix for both, implemented directly in the codebase (no code changes were speculative or left as recommendations only), plus the exact methodology and citations behind every weight now in use.

Where every weight now lives: a single new file, `services/priorityConfig.js`, computed by a second new file, `services/ahp.js` (a from-scratch implementation of the Analytic Hierarchy Process, not just the label). Every other file in the engine imports its weights from there. Nothing is duplicated or redefined elsewhere anymore.

2 Correctness fixes

2.1 The engine's own weighted score was being discarded

What was wrong: `LocationPriorityService.calculateComprehensivePriority()` — the method that actually produces a submitted complaint's stored priority — called `calculateLocationPriority()`, which does all the work of weighting proximity by facility-type importance, distance decay, density/diversity bonuses, and complaint-type relevance. It then *ignored that result* and instead summed the raw *count* of nearby places into an unrelated saturation curve. A location with 10 banks and 0 hospitals scored identically to one with 10 hospitals and 0 banks in the number that was actually stored.

BEFORE

```
infrastructureScore = 100 ×  
(1-e-totalFacilities/5)
```

— raw facility count only, hospital weight (0.9) computed elsewhere and never used here.

AFTER

```
infrastructureScore =  
locationPriority.priorityScore × 100
```

— the actual AHP-weighted, distance-decayed, bonus-adjusted proximity score.

File: `services/LocationPriorityService.js`, `calculateComprehensivePriority()`.

2.2 Three different, disagreeing formulas existed for the same score

Citizens previewing their complaint's priority (`/api/location-priority/calculate`, called from the submit-complaint screens) received a score from the fully-weighted formula. The score actually stored on the complaint (`routes/complaints.js`, via `POST /api/complaints/submit`) went through a wrapper that tried the same weighted formula first, but silently fell back to a *second, independent* formula (`location×0.6 + image×0.4`) whenever the first one threw — a formula whose own adjacent comment claimed 50%/40% weights, not 60/40. Which formula produced a citizen's stored score depended on whether a Google Places API call happened to succeed at that moment.

Fix: the fallback formula was deleted. There is now exactly one formula (`LocationPriorityService.calculateComprehensivePriority`, using the weights in Section 3), called identically by every entry point: complaint submission, the `/calculate-priority` preview, and

`/api/location-priority/comprehensive` . If the underlying calculation genuinely cannot run (no location data, or an exception), the system now returns an explicitly-labeled category-tier-only estimate (`degraded: true` in the response) rather than silently substituting a different, undocumented formula.

2.3 A fully-built feature was silently dead

`assessFacilityImportance()` was fully implemented (boosts hospitals/fire/police 30%, schools/government 10%) but its output was never attached to the facility objects it was meant to score — so `calculateFacilityScore()` 's importance multiplier always read the default value of 1.0. **Fix:** wired `assessFacilityImportance()` into `queryGooglePlaces()` 's result mapping, and its bump is now a function of the same AHP facility weights (Section 3.1) instead of independent numbers.

2.4 A fragile, redundant HTTP call to the server's own port

`calculateComprehensivePriority()` computed its emotion-score component by making an HTTP POST to `http://localhost:3001/api/emotion/analyze` — a hardcoded port, adding network latency and a failure mode (wrong port, service not yet listening) for logic that was already available in-process. It also meant the emotion score baked into a complaint's priority could be computed by different code than the `emotion_score` stored alongside it (which comes from `EmotionAnalysisService` directly). **Fix:** replaced the HTTP call with a direct call to the same `emotionService.analyzeEmotion()` used everywhere else.

2.5 Inconsistent priority-level thresholds

Three different threshold sets existed for turning a 0–1 score into a CRITICAL/HIGH/MEDIUM/LOW label: a 5-band version in `LocationPriorityService` and two different 4-band versions (one missing the MINIMAL band, another with different band edges) in `routes/complaints.js` and `routes/locationPriority.js` . All three now delegate to one function, `priorityConfig.getPriorityLevel()` .

2.6 A fairness bug in non-English emotion scoring

`EmotionAnalysisService` contained three separate, partially-stacked mechanisms that forced a non-zero minimum urgency/concern/frustration score onto Hindi/Tamil/Telugu complaint text whenever keyword matching found nothing — regardless of what the text actually said. In effect, an ordinary, non-urgent complaint written in Hindi or Tamil was guaranteed a higher emotion score than the identical complaint written in English, which has no keyword-floor. This systematically biased priority (and therefore triage order) by language.

Verified before fixing: tested `emotionService.analyzeEmotion('सड़क में गड्ढा है', 'pothole')` (Hindi: "there is a pothole in the road", no urgency language) — scored 0.40 concern / 0.35 urgency under the old floor logic purely because it was Hindi. After the fix it scores 0 (matching the equivalent mundane English text), while a genuinely urgent Hindi complaint ("**बड़ा गड्ढा है, बहुत खतरनाक है, दुर्घटना हो सकती है, तुरंत ठीक करें**") — "there is a big pothole, very dangerous, an accident could happen, fix it immediately") still correctly scores 0.57, driven by its actual urgency keywords, not a floor.

Fix: all three floor mechanisms were removed (`convertSentimentToEmotions` , `analyzeWithKeywords` , `analyzeWithEnhancedKeywords`). The legitimate corrections that remain are content-based: merging keyword analysis of a provided English translation, and re-scoring text the sentiment model likely misclassified using the same language's own keyword detector — both still gated on the text actually containing signal, never an unconditional floor.

2.7 Three disjoint, disagreeing category tables

The codebase had three independent category→score/label tables for the same idea ("how urgent is this type of complaint"): `EmotionAnalysisService` 's ~30-category multiplier table, `routes/complaints.js` 's ~15-category fallback-priority table, and its separate ~15-category importance-label table — using different category names for the same concept (`flooding` vs `road_waterlogging` , `garbage` vs `garbage_dumping`), several of which could not even occur given this app's actual 8-class complaint taxonomy. **Fix:** consolidated into one `CATEGORY_TIER_MAP` (Section 3.5) covering the union of every category name found anywhere in the codebase, mapped to 6 documented tiers.

2.8 Dead and duplicate files removed

Four files contained complete, unused, independently-diverging copies of scoring logic — confirmed via repository-wide search to be imported nowhere and not mounted by `server.js` — and were deleted rather than left as a latent risk of being wired in by mistake:

- `services/FallbackLocationService.js` — an OSM/Overpass-based location scorer with its own hand-typed facility weights and a 1 km-only distance cap.
- `routes/complaints-fixed.js` — an unmounted alternate complaints route with its own copies of the fallback-priority and category-importance tables.
- `routes/emotion-fixed.js` , `routes/emotion.js.backup` — unmounted alternate emotion routes.

3 How every weight is now derived

Every weight below is computed by `services/ahp.js` from a documented 1–9 criticality judgment via the **Analytic Hierarchy Process** (AHP), the standard method for turning subjective-but-structured expert judgment into defensible numeric weights [1] [2]. AHP works by having a decision-maker score how much more important one criterion is than another on Saaty's 1–9 intensity scale, assembling those judgments into a pairwise comparison matrix, and deriving normalized weights from it — along with a **Consistency Ratio (CR)** that flags self-contradictory judgments (e.g. $A > B$, $B > C$, but $C > A$). Saaty's accepted threshold is $CR \leq 0.10$ [3].

Honesty about what this is and isn't: the weights below replace unexplained literals with a documented, re-runnable, internally-consistent derivation — they are *not* claimed to be empirically optimal. This system has no historical resolution-outcome data yet (time-to-resolve, admin overrides, escalation) to fit weights by regression, which is the stronger method used in published 311-triage research [9] [10]. AHP is used here as the rigorous, auditable interim substitute, and Section 5 documents exactly which numbers should be the first candidates for that data-driven recalibration once outcome history exists.

3.1 Facility-type weights (proximity to critical infrastructure)

Criterion: if a reported hazard (pothole, downed line, flooding, structural damage) occurs near this facility and municipal response is delayed, how much does that increase risk to life or disruption of essential/emergency services? Hospital and fire station scores are grounded in NFPA 1710, the national U.S. standard setting a 4-minute engine travel-time target and an 8-minute ALS EMS arrival target [4] [5] — the two facility types whose response time is directly, externally standardized.

Facility type	Criticality (1–9)	AHP weight	Rationale
Hospital	9	0.1915	Concentrates the most immobile/vulnerable population; NFPA 1710 8-min ALS target [4].
Fire station	9	0.1915	NFPA 1710 4-min engine travel-time target; primary responder to this app's own hazard classes [4].
Police	7	0.1489	Scene-control first responder; secondary to fire/EMS for physical hazards.
School	6	0.1277	Vulnerable, low-autonomy population; supervised, so less acute than hospital/fire/police.
Transit station	5	0.1064	High throughput — a hazard here affects many people and can itself cause accidents.
Pharmacy	5	0.1064	Medicine access for the sick/elderly.
Government office	4	0.0851	Service continuity; not life-safety-critical.
Bank	2	0.0426	Financial continuity only; no direct life-safety link.

Consistency Ratio: `CR = 0.000` ✓ **consistent** (≤ 0.10) — computed by `services/ahp.js`, verified by re-running it against the live config (see Section 6).

3.2 Top-level signal weights (infrastructure / image / emotion / votes)

Criterion: *how reliable is this signal as evidence of real, severe civic infrastructure risk requiring prioritized response?*

Signal	Criticality (1–9)	AHP weight	Rationale
Image evidence (SAM3 detection confidence)	9	0.4286	Most direct, hardest-to-fabricate evidence the hazard physically exists and how severe it looks.
Infrastructure proximity	7	0.3333	Objective and geodata-verifiable, tied to consequence-of-delay, but doesn't confirm the complaint is genuine.
Text emotion	3	0.1429	Self-reported, comparatively easy to inflate; still carries information the other signals miss.
Community votes	2	0.0952	Most game-able and a lagging signal — a fresh report has had no time to accumulate votes.

Consistency Ratio: `CR = 0.000` ✓ **consistent**. Prior values (undocumented): infrastructure 0.40 / image 0.30 / emotion 0.20 / votes 0.10.

3.3 Emotion sub-weights

Emotion	Criticality (1–9)	AHP weight	Rationale
Urgency	9	0.4737	Direct textual signal of physical danger/emergency.
Concern	5	0.2632	Risk-relevant (safety, vulnerable groups) but less direct than explicit urgency language.
Anger	3	0.1579	Mostly reflects dissatisfaction with response, weakly tied to physical severity.
Frustration	2	0.1053	Reflects an unresolved backlog — a service-quality signal, not current severity.

Consistency Ratio: `CR = 0.000` ✓ **consistent**. Prior values (undocumented): urgency 0.4 / anger 0.3 / concern 0.2 / frustration 0.1 — note concern and anger are swapped in rank versus the AHP result.

3.4 Text-authenticity blend

Component	Criticality (1–9)	AHP weight	Rationale
Category match	7	0.6364	Directly detects a mismatched/fabricated report — the strongest available signal.
Text quality	4	0.3636	Detects spam/gibberish, but doesn't confirm topical relevance.

This independently-elicited split (0.636/0.364) landed within 0.01 of the pre-existing hand-picked 0.65/0.35 — that blend is effectively unchanged, now with a documented rationale instead of none.

3.5 Complaint-category urgency tiers

Replaces ~30 independently hand-picked per-category multipliers (which also contained several category names that could never occur, per Section 2.7) with 6 documented tiers, converted to a multiplier by one linear formula: $\text{multiplier} = 1.0 + (\text{tierScore}/9) \times 0.9$, matching the original [1.10, 1.90] range.

Tier	Score	Multiplier	Example categories
Life-threatening	9	1.90×	fallen_electric_pole, gas_leak, fire_hazard, building_collapse
Public safety	7	1.70×	women_safety, road_safety, traffic_signal, broken_streetlight
Infrastructure integrity	6	1.60×	pothole, road_waterlogging, concrete_structure_damage, fallen_tree
Basic services	4	1.40×	garbage_dumping, stray_cattle, water_supply, power_outage
Environmental	3	1.30×	air_pollution, water_pollution, illegal_dumping
Civic / administrative	1	1.10×	others, park_maintenance, document_issue, general

3.6 Distance decay

Facility proximity previously used a linear falloff, $1 - \text{distance}/\text{radius}$, which has an artificial hard zero exactly at the search-radius edge and no support in the literature for a linear shape. It is replaced with a **negative-exponential decay** function, $\text{score} = e^{-\ln(2) \cdot \text{distance}/\text{halfDistance}}$, the standard functional form in gravity-based spatial-accessibility research, including the Two-Step Floating Catchment Area (2SFCA) method literature [6] [7]. The half-distance parameter (where score = 0.5) defaults to each facility type's configured search radius ÷ 2 — an explicit, documented convention, not itself derived from external data (see Section 5).

3.7 Priority-level thresholds

Unified to one 5-band **equal-interval classification** (CRITICAL ≥0.8, HIGH ≥0.6, MEDIUM ≥0.4, LOW ≥0.2, MINIMAL ≥0.0) across every file. Equal-interval is a standard, named data-classification method [8]; it is an explicit interim convention pending real score-distribution data (see Section 5).

4 Verification

Every file was syntax-checked (`node --check`) after editing. The AHP module was executed directly to confirm all four weight sets are internally consistent ($CR = 0$ for all, since each was constructed from directly-elicited ratio judgments — see `services/ahp.js` docstring for why $CR = 0$ is expected here and what it does and doesn't prove). The full `calculateComprehensivePriority()` pipeline was run end-to-end against a live test case (pothole complaint, image confidence 0.8, urgent English description, 3 votes, no Places API key configured) and produced a correctly-weighted breakdown (infrastructure 0.00×0.333 , image $80.00 \times 0.429 = 34.29$, emotion $100.00 \times 0.143 = 14.29$, votes $45.12 \times 0.095 = 4.30 \rightarrow 52.87\%$ MEDIUM) with no unhandled exceptions, confirming the missing-API-key and zero-facility code paths degrade gracefully. The non-English emotion-scoring fairness fix was verified with paired before/after test cases (Section 2.6).

5 What remains an explicit default, not a cited number

In the interest of not overstating what is proven: the following specific numeric choices are transparent, documented engineering conventions rather than externally-sourced or empirically fit values, and are the recommended first candidates for recalibration once real outcome data exists:

- The distance-decay half-distance default (facility radius $\div 2$) — the decay *function shape* is cited (Section 3.6), its specific parameter is not.
- The complaint-type $\leftrightarrow$ relevant-facility-type relevance mapping (e.g. "potholes matter more near hospitals/schools/transit") — a retained domain judgment, not re-derived.
- The density/diversity bonus caps (0.25 / 0.15) and saturation-curve scale constants — preserved from the original design; only their shape (smooth curve vs. arbitrary staircase) changed.
- Every 1–9 AHP criticality score itself (Section 3) — the *method* for turning judgment into weights is cited and auditable; the judgments are a documented starting point meant to be revisited with real municipal stakeholders or outcome data, not a claim of empirical proof.

Recommended next step: begin logging resolution outcomes (time-to-resolve, admin overrides, escalations) now. Once several months of history exist, refit the weights in Section 3 by regression against those outcomes — the approach used in published 311-triage systems [9] [10] — and replace the equal-interval priority thresholds (Section 3.7) with data-driven quantile or Jenks natural-breaks thresholds [8].

6 Files changed

- **New** — `services/ahp.js` : AHP weight/consistency computation
- **New** — `services/priorityConfig.js` : single source of truth for every weight
- **Modified** — `services/LocationPriorityService.js`
- **Modified** — `services/EmotionAnalysisService.js`
- **Modified** — `services/TextAuthenticityService.js`
- **Modified** — `routes/complaints.js`
- **Modified** — `routes/locationPriority.js`
- **Removed** — `services/FallbackLocationService.js` (dead, divergent)
- **Removed** — `routes/complaints-fixed.js` (unmounted duplicate)
- **Removed** — `routes/emotion-fixed.js` (unmounted duplicate)
- **Removed** — `routes/emotion.js.backup` (unmounted duplicate)

No changes were made outside `UrbanPulse_Backend` ; the frontend was not modified (it already consumes whichever score the backend returns, so the citizen-preview/stored-score inconsistency in Section 2.2 is fixed purely by the backend consolidation).

7 Sources

1. **Saaty, T.L. (1980).** *The Analytic Hierarchy Process*. McGraw-Hill. — Summarized in: [1000minds, "What is the Analytic Hierarchy Process \(AHP\)?"](#)
2. **AHP methodology primer** — [International Institute for Environment and Development \(IIED\), "Analytic hierarchy process \(AHP\)"](#)
3. **Saaty, T.L. (2000).** *Fundamentals of Decision Making and Priority Theory*. RWS Publications. Consistency Ratio methodology and Random Index table — see also ["A Note on the Computation of the Mean Random Consistency Index of the AHP," Theory and Decision \(Springer\)](#)
4. **NFPA 1710** response-time standard (fire suppression / EMS deployment) — ["NFPA 1710 Response Times: What You Need To Know," Emergent](#)
5. **DC Fire & EMS** — ["Evaluating Response Times," dc.gov](#)
6. **Gravity-model spatial accessibility / distance decay** — ["Gravity models for potential spatial healthcare access measurement: a systematic methodological review," PMC](#)
7. **Enhanced Two-Step Floating Catchment Area (E2SFCA) method** — [Luo & Qi, "An enhanced two-step floating catchment area \(E2SFCA\) method," Health & Place \(ScienceDirect\)](#)
8. **Data classification methods** (equal interval, Jenks natural breaks) — [Esri ArcGIS Pro documentation, "Data classification methods"](#)
9. **Municipal 311 complaint triage / ML prioritization** — ["Scaling the Queue: Reinforcement Learning for Equitable Call Classification Capacity in NYC Municipal Complaint Systems," arXiv](#)
10. **311 demand prediction** — [Wang et al., "Predicting demand for 311 non-emergency municipal services: An adaptive space-time kernel approach," Computers, Environment and Urban Systems \(ScienceDirect\)](#)